

Logistic Regression Hyperparameters

Contents

1. Overview	2
2. Regularisation	2
2.1 The type of penalty	2
2.2 The strength: C	3
3. The solver	4
4. Training settings	5
5. Other settings	5
5.1 class_weight	6
6. Multi-class settings	6
7. An interactive playground	6
8. Summary	8
9. Key terms	8

Where this fits: Pipeline map step 10 (Tune). See the [Course map](#).



- **Builds on:** Overfitting ([Note 61](#)).
- **Leads to:** Grid and random search ([Note 91](#)); Random forest ([Note 108](#)); AdaBoost ([Note 115](#)); Optuna (Video 134, coming).

1. Overview

Key point: The settings that matter most are the regularisation (C and the L1/L2 mix), the solver, `max_iter` and `class_weight`. The rest can usually stay at their defaults.

scikit-learn's `LogisticRegression` has about fifteen hyperparameters. This Note explains what each one does, which ones are worth tuning, and shows their effect on real data. An interactive app at the end lets you change them and watch the decision boundary move.

Extra: scikit-learn 1.8 changed how the penalty is chosen. The old `penalty="l1" / "l2" / "elasticnet" / "none"` setting is deprecated. The L1/L2 mix is now set with `l1_ratio` alone, and “no penalty” is `C=np.inf`. Older code and tutorials use `penalty`; this Note shows both.

2. Regularisation

Key point: By default, logistic regression adds an L2 (Ridge) penalty. C is the inverse of its strength: a small C means strong regularisation.

2.1 The type of penalty

Key point: `l1_ratio = 0` is L2 (Ridge), `1` is L1 (Lasso), in between is Elastic Net; `C = inf` switches the penalty off.

The same three penalties as in the regularised regression Notes are available:

Wanted	New way (1.8+)	Old way
L2 (Ridge), the default	<code>l1_ratio=0</code>	<code>penalty="l2"</code>
L1 (Lasso)	<code>l1_ratio=1</code>	<code>penalty="l1"</code>
Elastic Net	<code>0 < l1_ratio < 1</code>	<code>penalty="elasticnet"</code>
no penalty	<code>C=np.inf</code>	<code>penalty=None</code>

On the breast-cancer data (569 tumours, 30 measurements, inputs standardised, 5-fold cross-validation):

Setting	CV accuracy	Non-zero coefficients
no penalty	0.956	30
L2, $C = 1$ (default)	0.981	30
L1, $C = 1$	0.970	16
Elastic Net, <code>l1_ratio=0.5</code> , $C = 1$	0.974	26

Without a penalty the model overfits slightly (and the solver struggles to converge). As in the Lasso Note, L1 sets many coefficients to exactly zero: here it keeps 16 of the 30 measurements.

2.2 The strength: C

Key point: C is $1/\lambda$. Smaller C : smaller coefficients and simpler boundaries; too small underfits. Larger C : closer to no penalty.

In the Ridge and Lasso Notes, the penalty strength was λ (alpha). Logistic regression uses its inverse, $C = 1/\lambda$, so the direction is reversed: **a smaller C means stronger regularisation**. The default is $C=1.0$.

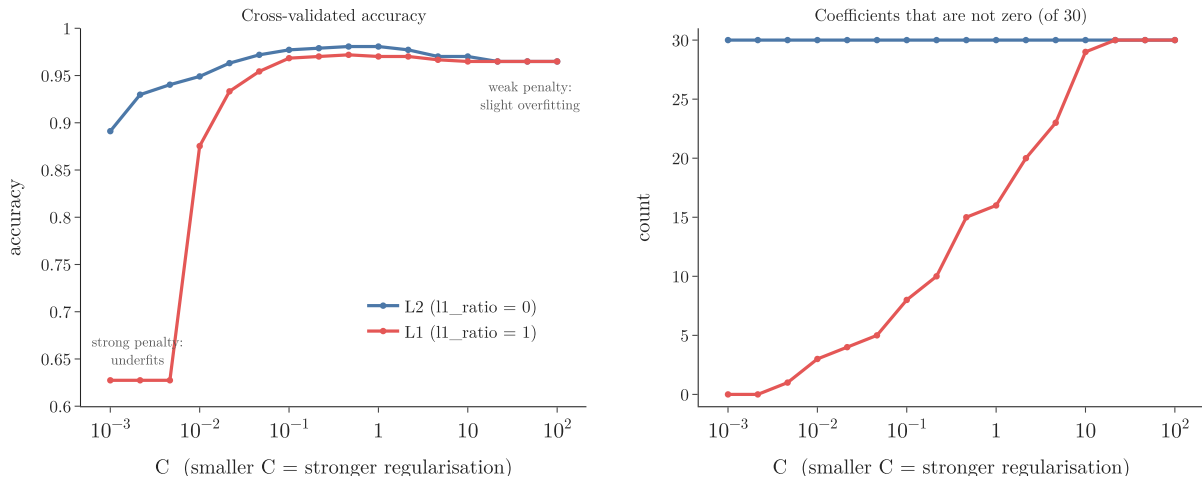


Figure 1: Cross-validated accuracy and number of non-zero coefficients against C

- With **L2** (blue), accuracy peaks around $C = 0.3$ to 1 (0.981). Very small C squeezes the coefficients so hard that accuracy falls to 0.89; very large C slightly overfits.
- With **L1** (red), small C sets more and more coefficients to zero (right panel). At $C = 0.001$ every coefficient is zero and the model just predicts the most common class (accuracy 0.63).

C	CV accuracy (L2)	Largest coefficient
0.001	0.891	0.095
0.1	0.977	0.597

C	CV accuracy (L2)	Largest coefficient
1	0.981	1.321
100	0.963	7.696

C is the hyperparameter most worth tuning, typically by trying values spaced by factors of 10 with cross-validation (the grid search Note).

3. The solver

Key point: The solver is the optimisation method. Each one supports only some penalties; lbfgs, the default, does L2 or no penalty.

Our own gradient descent (the gradient descent Note) was one way to minimise the log loss. scikit-learn offers several faster methods, called **solvers**:

Solver	L2	L1	Elastic Net	None	Notes
lbfgs (default)	yes			yes	good general choice
newton-cg	yes			yes	
newton-cholesky	yes			yes	fast when rows far outnumber columns
liblinear	yes	yes			small data; one-vs-rest for several classes
sag	yes			yes	large data; needs scaled inputs
saga	yes	yes	yes	yes	large data; the only one for Elastic Net

Asking for a combination that is not supported raises an error, for example `LogisticRegression(l1_ratio=1, solver="lbfgs")` fails with “Solver lbfgs supports only ‘l2’ or None penalties”. For L1 use `liblinear` or `saga`; for Elastic Net use `saga`.

4. Training settings

Key point: `max_iter` and `tol` control when the solver stops; raise `max_iter` if a `ConvergenceWarning` appears.

Setting	Default	Meaning
<code>max_iter</code>	100	the maximum number of solver iterations (like epochs)
<code>tol</code>	0.0001	stop when an iteration improves the result by less than this
<code>warm_start</code>	False	if True, a second <code>fit</code> starts from the previous coefficients instead of from scratch
<code>verbose</code>	0	print progress while training
<code>n_jobs</code>	None	CPU cores to use; only helps in a few cases (<code>-1</code> = all)
<code>random_state</code>	None	seed for solvers that shuffle the data (<code>sag</code> , <code>saga</code> , <code>liblinear</code>)

If scikit-learn prints a `ConvergenceWarning`, the solver stopped at `max_iter` before reaching the minimum. The fixes are to standardise the inputs, raise `max_iter`, or both. Lowering `tol` is rarely needed.

5. Other settings

Key point: `class_weight`=“balanced” helps on imbalanced classes; `fit_intercept`, `dual` and `intercept_scaling` rarely need changing.

Setting	Default	Meaning
<code>class_weight</code>	None	"balanced" gives rare classes more weight in the loss
<code>fit_intercept</code>	True	learn the intercept w_0 ; almost always leave it on
<code>dual</code>	False	an alternative formulation for <code>liblinear</code> with L2, useful only when columns outnumber rows
<code>intercept_scaling</code>	1	a technical setting for <code>liblinear</code> only

5.1 class_weight

Key point: On a dataset with 5% positives, “balanced” raised recall from 0.79 to 0.88 with no loss of precision.

On imbalanced data (the accuracy Note), the loss is dominated by the common class, so the model tends to neglect the rare one. `class_weight="balanced"` multiplies each row’s loss by a weight inversely proportional to its class’s frequency, so mistakes on the rare class count more.

On a synthetic dataset with 5% positives (33 in a 600-row test set):

class_weight	Recall	Precision
None	0.788	1.000
"balanced"	0.879	1.000

The weighted model finds three more of the 33 rare cases. In general, weighting raises recall and may lower precision, so check both.

6. Multi-class settings

Key point: With three or more classes, LogisticRegression uses softmax automatically; one-vs-rest needs OneVsRestClassifier.

Older versions had a `multi_class` setting ("ovr", "multinomial", "auto"). It has been removed: with three or more classes, all solvers except `liblinear` now fit the softmax model of the previous Note. For one-vs-rest, wrap the model: `OneVsRestClassifier(LogisticRegression())`.

7. An interactive playground

Key point: `app.py` lets you change the dataset, penalty, C, solver, `l1_ratio` and `max_iter` and see the decision regions update.

The folder of this Note contains `app.py`, a small Dash app. Run `python app.py` and open `http://127.0.0.1:8050`.

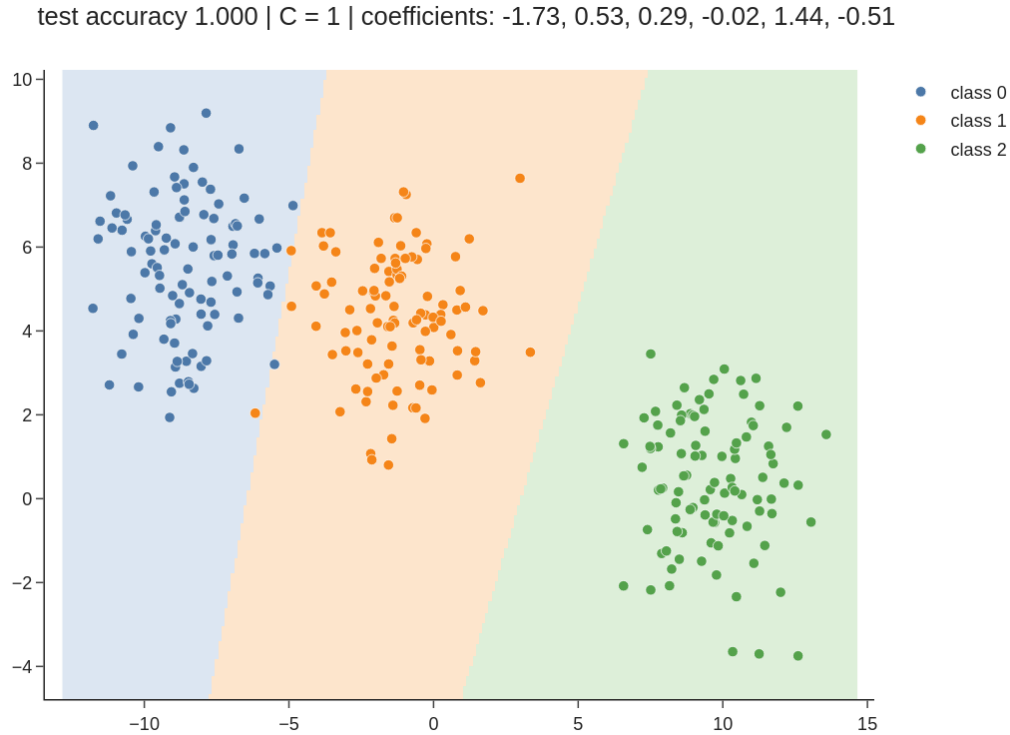


Figure 2: The playground on the three-class dataset with the default settings

Things to try:

- **C** from 3 down to -3 (that is, 10^3 to 10^{-3}) with L2: the boundaries stay straight but the coefficients shrink and the model becomes less sure.
- **L1 with small C**: one of the two coefficients drops to exactly 0, and the boundary becomes vertical or horizontal (it uses only one input).
- **Penalty L1 with solver lbfgs**: the app shows the “not allowed” error.
- **max_iter = 10**: the solver stops early and the boundary is not yet in place.

8. Summary

Hyperparameter	Default	Tune it?
<code>C</code>	1.0	yes: the most important; smaller = stronger regularisation
<code>l1_ratio</code> (old <code>penalty</code>)	0 (L2)	sometimes: 1 for feature selection, between for Elastic Net
<code>solver</code>	<code>lbfgs</code>	only to allow L1/Elastic Net or for very large data
<code>max_iter</code>	100	raise it if a <code>ConvergenceWarning</code> appears
<code>class_weight</code>	None	" balanced " for imbalanced classes
others		rarely

9. Key terms

Term	Meaning
<code>C</code>	The inverse of the regularisation strength in <code>LogisticRegression</code> ; smaller <code>C</code> means stronger regularisation
<code>l1_ratio</code>	The share of the penalty that is L1: 0 is Ridge, 1 is Lasso, in between is Elastic Net
<code>Solver</code>	The optimisation method used to find the coefficients
<code>lbfgs</code>	The default solver of <code>LogisticRegression</code> ; supports L2 or no penalty
<code>saga</code>	A stochastic solver that supports every penalty, including Elastic Net
<code>ConvergenceWarning</code>	A warning that the solver stopped at <code>max_iter</code> before reaching the minimum
<code>class_weight</code>	A setting that weights each class's mistakes in the loss; "balanced" helps rare classes
<code>Dash</code>	A Python library for building interactive web apps with Plotly charts